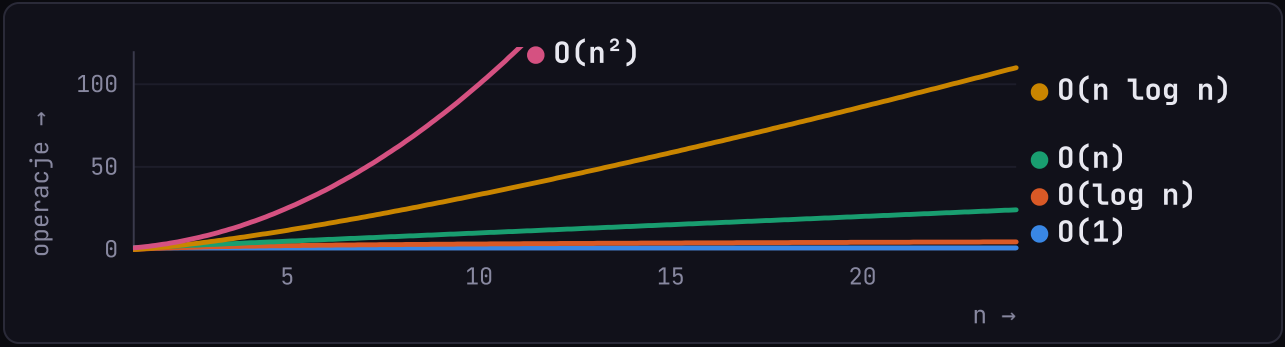
// T-OLIMPIANS — PREZENTACJA DO LEKCJI

Złożoność czasowa — krok po kroku

Jak przewidzieć, czy program zmieści się w limicie czasu — bez uruchamiania go. Operacje i sekunda, notacja dużego O, po kolei O(1), O(n), O(n²), logarytm i O(log n), O(n log n), O(2ⁿ) i O(n!), reguły liczenia, dopasowanie złożoności do ograniczeń z treści oraz brut i wzorcówka.

C++ · LEKCJA 17 · ZŁOŻONOŚĆ CZASOWA · BEZ ZNAJOMOŚCI ALGORYTMÓW



PLAN PREZENTACJI

01 Po co liczyć czas?
02 Operacje i sekunda
03 Liczymy operacje
04 Notacja dużego O
05 O(1) — czas stały
06 O(n) — jedna pętla
07 O(n²) i więcej pętli
08 Logarytm
09 O(log n)
10 O(n log n)
11 O(2ⁿ) i O(n!)
12 Porównanie złożoności
13 Jak liczyć złożoność?
14 Limit czasu a n
15 Brut i wzorcówka
16 Sprawdź się i ściąga

01 Każde zadanie ma limit czasu

Domino Fibonacciego

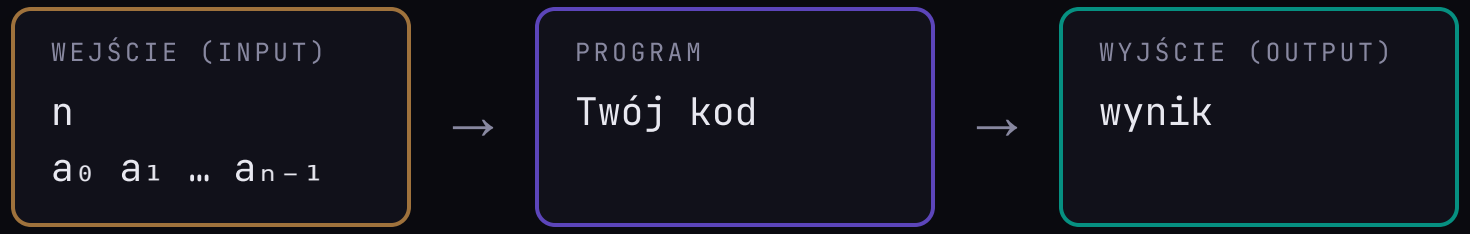
XIX OIJ, zawody II stopnia · 1 marca 2025

kod	fib
limit czasu	1 s
limit pamięci	64 MB

Na górze treści każdego zadania olimpijskiego są **limity**. Przykład z notatek: zadanie z Olimpiady Informatycznej Juniorów.

1. **Limit czasu** — ile sekund może działać Twój program na jednym teście.
2. **Limit pamięci** — ile pamięci może zająć (tym zajmiemy się innym razem).
3. W **99% zadań** limit czasu to **1 sekunda** (z notatek).

01 Jak sprawdzarka ocenia program?



1. Sprawdzarka uruchamia Twój program na wielu **testach** — małych i bardzo dużych.
2. Do każdego testu podaje **wejście** i mierzy, **ile czasu** program liczy.
3. Jeśli program liczy dłużej niż limit, test jest oblany: **„przekroczono limit czasu”** — 0 punktów za ten test, nawet gdy wynik byłby dobry!

Dlatego nie wystarczy, że program jest **poprawny**. Musi być też **wystarczająco szybki**.

01 Ten sam wynik, zupełnie inny czas

Dwa programy liczą to samo: sumę **1 + 2 + 3 + … + n**.

PROGRAM A — PĘTLA

```
long long n, suma = 0;
cin >> n;
for(long long i = 1; i <= n; i++){
    suma += i;
}
cout << suma << endl;
```

PROGRAM B — WZÓR

```
long long n;
cin >> n;
cout << n * (n + 1) / 2 << endl;
```

Dla n = 100 oba wypisują **5050**.

n	program A	program B
100	błyskawicznie	błyskawicznie
10^9 (miliard)	kilka sekund — **× za wolno**	błyskawicznie — **✓ zmieści się**

Oba dają ten sam wynik. Ale dla dużego n program A nie zmieści się w 1 sekundzie. Dziś nauczymy się **przewidywać** takie rzeczy — bez uruchamiania programu. To jest **złożoność czasowa**.

02 Co to jest operacja?

Komputer wykonuje program krok po kroku. Jeden prosty krok to **operacja**. Przykłady:

Kod	Co robi	Ile operacji
`x = 5;`	przypisanie	1
`a + b`	dodawanie (też -, *, /, %)	1
`i < n`	porównanie	1
`i++`	zwiększenie o 1	1
`lista[i]`	odczyt z tablicy / vectora	1
`cin >> x;` · `cout << x;`	wczytanie / wypisanie jednej liczby	ok. 1

Nie liczymy operacji **dokładnie** co do jednej — to niemożliwe i niepotrzebne. Wystarczy nam **przybliżenie**: czy to około tysiąc, milion, czy miliard operacji.

02 Przypomnienie: potęgi dziesiątki

Duże liczby zapisujemy jako potęgi dziesiątki. 10^k to **jedynka i k zer**:

Zapis	Liczba	Słownie
10^1	10	dziesięć
10^3	1 000	tysiąc
10^5	100 000	sto tysięcy
10^6	1 000 000	milion
10^8	100 000 000	sto milionów
10^9	1 000 000 000	miliard
$5 \cdot 10^8$	500 000 000	pięćset milionów
10^{18}	1 i 18 zer	trylion

Mnożenie potęg: dodajemy wykładniki. $10^5 \cdot 10^5 = 10^{10}$, a $10^3 \cdot 10^3 = 10^6$.

02 Ile operacji w 1 sekundę?

Komputer wykonuje około 10^8 prostych operacji na sekundę.

Z notatek: maksymalnie ok. **$5 \cdot 10^8 = 500\,000\,000$** operacji (procesor 2 GHz). Ale lepiej liczyć ostrożnie — operacje bywają wolniejsze, a sprawdzarka może być słabsza:

Liczba operacji	Czy zmieścimy się w 1 s?
10^6 — milion	✓ **zmieści się** — z ogromnym zapasem
10^7 — 10 milionów	✓ **zmieści się** — na pewno (z notatek)
10^8 — 100 milionów	✓ **zmieści się** — zwykle tak
$5 \cdot 10^8$	**granica** — ryzykownie
10^9 — miliard	× **za wolno**
10^{10}	× **za wolno** — ok. 100 sekund

02 Ile sekund zajmie program?

Szybki szacunek: **liczba operacji ÷ 10^8** ≈ liczba sekund.

Operacje	Czas	Werdykt
10^6	ok. 0,01 s	✓ zmieści się
10^8	ok. 1 s	✓ zmieści się (na styk)
10^9	ok. 10 s	× za wolno
10^{10}	ok. 100 s — prawie 2 minuty	× za wolno
10^{12}	ok. 3 godziny	× za wolno
10^{18}	ok. 300 lat	× za wolno

Różnica między „mieści się” a „liczyłby się latami” to często tylko **jedna pętla więcej**. Za chwilę zobaczymy dlaczego.

03 Program bez pętli

KOD Z NOTATEK (Z DROBNYM DOPISKIEM)

```
int n;              // 1 operacja
string napis;       // 1 operacja
cin >> n;           // 1 operacja
cout << n * 2;      // ok. 2 operacje
```

n	operacji
10	ok. 5
1 000	ok. 5
10^9	ok. 5

Każda linijka wykona się **dokładnie raz**. Nieważne, czy n to 10, czy miliard — program robi zawsze **tyle samo** operacji.

03 Program z pętlą — schemat z notatek

INPUT

N

→

PROGRAM

```
for(int i = 0; i < n; i++){
    cout << i << endl;
}
```

→

OUTPUT

```
0
1
2
⋮
n-1
```

1. Program wczytuje **n**.
2. Pętla wykonuje się **n razy** — dla i = 0, 1, 2, …, n−1.
3. W każdym obrocie jest jedno `cout` — więc razem **n** wypisań.

Tutaj liczba operacji **zależy od n**. Im większe wejście, tym dłużej działa program.

03 Liczymy dla różnych n

```
for(int i = 0; i < n; i++){
    cout << i << endl;
}
```

n	ile razy cout?	czas (÷ 10^8)	werdykt
10	10	nic	✓ **zmieści się**
1 000	1 000	nic	✓ **zmieści się**
10^5	10^5	0,001 s	✓ **zmieści się**
10^8	10^8	ok. 1 s	✓ **zmieści się** (na styk)
10^9	10^9	ok. 10 s	× **za wolno**

Wniosek: n razy większe wejście → n razy więcej operacji. Liczba operacji rośnie **tak samo jak n**.

03 Dokładnie czy w przybliżeniu?

```
for(int i = 0; i < n; i++){
    cout << i << endl;
}
```

Część	Ile razy
`int i = 0`	1
`i < n`	n + 1
`i++`	n
`cout << i`	n

Policzmy **dokładnie**: 1 + (n + 1) + n + n = **3n + 2** operacje.

n	3n + 2	samo n
10	32	10
1 000	3 002	1 000
10^6	ok. $3 \cdot 10^6$	10^6

Obie kolumny rosną **tak samo**: gdy n rośnie 10 razy, obie rosną ok. 10 razy. Liczenie „3n + 2" zamiast „n" nic nie zmienia w ocenie, czy program zdąży. Dlatego upraszczamy.

04 Notacja dużego O

O(…) mówi, jak szybko rośnie liczba operacji, gdy rośnie wejście.

Z notatek: **notacja dużego O pozwala obliczyć, ile operacji maksymalnie wykona nasz program**, zależnie od danych wejściowych.

Jak czytać?

`O(n)` czytamy „o od en”. W nawiasie piszemy przybliżoną liczbę operacji, używając n — rozmiaru wejścia.

Jak nazywać?

To, co jest w nawiasie, to **złożoność czasowa** programu. Np. „ten program ma złożoność O(n)”.

04 Przykłady z notatek

Złożoność	n = 10	n = 1000
$O(1)$	ok. 1 operacja	ok. 1 operacja
$O(n)$	ok. 10 operacji	ok. 1 000 operacji
$O(n^2)$	ok. $10^2 = 100$ operacji	ok. $1000^2 = 1\,000\,000$ operacji

Wstawiamy n do wyrażenia w nawiasie i dostajemy przybliżoną liczbę operacji. Przy n = 1000 widać już ogromną różnicę: tysiąc kontra milion.

04 Zasada 1: pomijamy stałe

Dokładnie	Zapisujemy	Dlaczego
$3n + 2$	$O(n)$	mnożnik 3 i dodane 2 pomijamy
$5n$	$O(n)$	mnożnik 5 pomijamy
$n / 2$	$O(n)$	połowa n to wciąż „rzędu n”
100	$O(1)$	stała liczba — nie zależy od n
$2n^2 + 7$	$O(n^2)$	mnożnik 2 i dodane 7 pomijamy

Stałe nie zmieniają tego, **jak** rośnie liczba operacji. 3n i n tak samo rosną 10 razy, gdy n rośnie 10 razy.

04 Zasada 2: zostaje najszybciej rosnący składnik

Gdy mamy sumę kilku części, np. **$n^2 + n$**, zostawiamy tylko tę, która rośnie najszybciej:

n	n^2	n	$n^2 + n$	ile to n w sumie?
10	100	10	110	ok. 9%
1 000	1 000 000	1 000	1 001 000	ok. 0,1%
10^5	10^{10}	10^5	ok. 10^{10}	prawie nic

Czyli: $n^2 + n \to$ **$O(n^2)$**. $n^2 + 5n + 100 \to$ **$O(n^2)$**. $n + 1000 \to$ **$O(n)$**.

04 Liczymy najgorszy przypadek

```
// czy w vectorze jest liczba 7?
int i = 0;
while(i < n && lista[i] != 7){
    i++;
}
```

Przypadek	Ile obrotów
7 na początku	0 — od razu
7 w środku	ok. n / 2
7 na końcu albo brak 7	**n**

Czasem program kończy wcześniej — zależy, jakie dostanie dane. Którą liczbę wpisujemy w O(…)?

Odpowiedź: zawsze **najgorszy przypadek** — największą liczbę operacji, jaką program może wykonać. Tutaj: **O(n)**.

Z notatek: „w najgorszym przypadku będziemy robić…". Sprawdzarka na pewno da test, w którym będzie najgorzej.

04 Złożoności z notatek — od najszybszej

Złożoność	Nazwa	Typowy kod	Rozdział
$O(1)$	stała	brak pętli zależnej od n	05
$O(\log n)$	logarytmiczna	dzielenie n na pół w pętli	08–09
$O(n)$	liniowa	jedna pętla do n	06
$O(n \log n)$	liniowo-logarytmiczna	pętla do n, a w niej dzielenie na pół	10
$O(n^2)$	kwadratowa	pętla w pętli	07
$O(n^3)$	sześcienna	trzy pętle w sobie	07
$O(2^n)$	wykładnicza	wszystkie podzbiory	11
$O(n!)$	silnia	wszystkie ustawienia (kolejności)	11

Omówimy je po kolei — każdą z kodem, tabelą i wykresem.

05 O(1) — czas stały

Z NOTATEK

```
int n;          // O(1) — tworzymy zwykłego int-a
string napis;   // O(1) — tworzymy zwykłego stringa
cin >> n;       // O(1)
```

Z notatek: **stała złożoność czasowa** — robimy stałą liczbę operacji (np. 5), nieważne czy input jest duży, czy mały.

n	operacji
100	ok. 5
10 000	ok. 5
10^9	ok. 5



05 Przykłady O(1)

Kod	Dlaczego O(1)
`lista[i]` · `lista[i] = 7;`	odczyt i zapis jednej szufladki — od razu, bez szukania
`lista.size()`	vector pamięta swój rozmiar — nie musi liczyć
`lista.push_back(x);`	dokłada jeden element na koniec (praktycznie zawsze O(1))
`if(a > b) cout << a;`	jedno porównanie i jedno wypisanie
`for(int i = 0; i < 100; i++)`	pętla zawsze 100 razy — **nie zależy od n**!
`n * (n + 1) / 2`	wzór — kilka działań, niezależnie od n

Uwaga: O(1) **nie** znaczy „dokładnie jedna operacja”. Może być 5 albo 100 operacji — ważne, że ich liczba **nie zależy od wejścia**.

05 Wzór zamiast pętli — O(n) kontra O(1)

PĘTLA — O(N)

```
long long n, suma = 0;
cin >> n;
for(long long i = 1; i <= n; i++){
    suma += i;
}
cout << suma << endl;
```

WZÓR — O(1)

```
long long n;
cin >> n;
cout << n * (n + 1) / 2 << endl;
```

n	pętla	wzór
100	100 obrotów	3 działania
10^6	10^6 obrotów	3 działania
10^9	10^9 — **× za wolno**	3 działania — ✓ **zmieści się**

Wzór na sumę 1 + 2 + … + n = **n · (n + 1) / 2** znasz z lekcji o vectorze (tak liczyliśmy podtablice). Dla n = 100: 100 · 101 / 2 = **5050**.

Zapamiętaj: zamiana pętli na wzór to częsty sposób na przyspieszenie programu.

06 O(n) — jedna pętla do n

Z NOTATEK

```
for(int i = 0; i < n; i++){   // n razy → O(n)
    cout << i << endl;        // O(1)
}
```

Z notatek: **ten for-loop wykona się n razy**, czyli liczba powtórzeń zależy od inputu (konkretnie od „n") → O(n).

> Wnętrze pętli to O(1). Pętla powtarza je n razy: **n · O(1) = O(n)**.



Wykres to **prosta linia**: dwa razy większe n → dwa razy więcej operacji.

06 O(n) — przykłady z poprzednich lekcji

```
for(auto &e : lista) cin >> e;
```
wczytywanie vectora

wczytanie n liczb

```
for(auto el : lista) suma += el;
```
suma

suma wszystkich elementów

```
if(el % 2 == 0) parzyste++;
```
zadania z logiką

w pętli: liczenie parzystych

```
if(el > maks) maks = el;
```
maksimum

w pętli: największa liczba

```
for(int i = 0; i < n; i++) cout << t[i];
```
lekcja o tablicach

wypisanie tablicy

Wszystko, co robi **jedno przejście** po tablicy albo vectorze, to O(n): każdy element odwiedzamy **raz**, robiąc przy nim stałą liczbę operacji → n · O(1) = O(n).

06 Dwie pętle po kolei — wciąż O(n)

Z NOTATEK

```
for(int i = 0; i < n; i++){   // O(n)
    cout << i << endl;
}
for(int j = 0; j < n; j++){   // O(n)
    cout << j << endl;
}
// razem: O(n) + O(n) = O(2n) = O(n)
```

n	operacji
10	20
1 000	2 000
10^6	$2 \cdot 10^6$

Pętle stoją **jedna po drugiej**, więc ich czasy **dodajemy**: n + n = 2n. Stałą 2 pomijamy (zasada 1) → **O(n)**.

06 Uwaga: liczy się, ile razy obróci się pętla

Pętla	Ile obrotów	Złożoność
`for(int i = 0; i < n; i++)`	n	`O(n)`
`for(int i = 0; i < n; i += 2)`	n / 2	`O(n)` — stałą 1/2 pomijamy
`for(int i = 0; i < 3 * n; i++)`	3n	`O(n)`
`for(int i = 0; i < 100; i++)`	100 — zawsze	`O(1)` — nie zależy od n!
`for(int i = 0; i < m; i++)`	m	`O(m)` — inna zmienna

Rada: nie patrz na samo słowo **for** — sprawdź, **ile razy** naprawdę się wykona.

07 Pętla w pętli

Z NOTATEK

```
for(int i = 0; i < n; i++){          // O(n)
    for(int j = 0; j < n; j++){      // O(n)
        cout << i << ' ' << j << endl;   // O(1)
    }
}
// złożoność całego bloku: O(n²)
```

WEJŚCIE: N = 3

```
3
```

NA EKRANIE: 9 LINIJEK

```
0 0
0 1
0 2
1 0
1 1
1 2
2 0
2 1
2 2
```

Dla n = 3 program wypisał **9 = 3 · 3** par.

07 Dlaczego n · n?

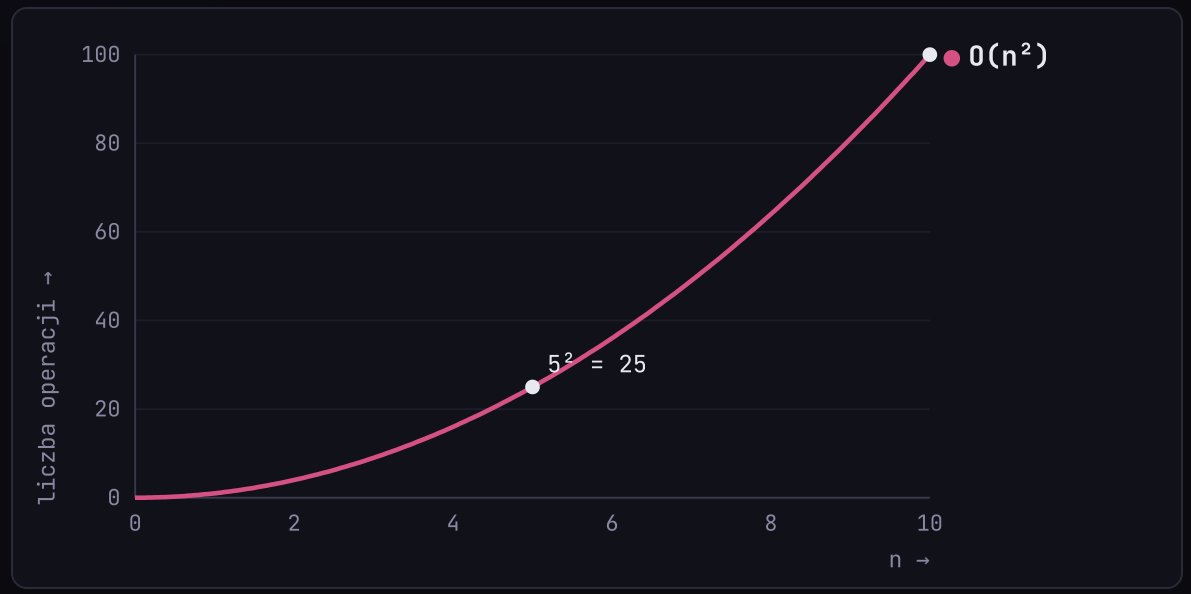
pary (i, j) dla n = 4

i \ j	0	1	2	3
0	0,0	0,1	0,2	0,3
1	1,0	1,1	1,2	1,3
2	2,0	2,1	2,2	2,3
3	3,0	3,1	3,2	3,3

1. Pętla zewnętrzna ustawia **i** — n różnych wartości (wiersze).
2. Dla **każdego** i pętla wewnętrzna obraca się **n razy** (kolumny).
3. Razem: n + n + … + n (n razy) = **n · n = n²**.

Pętle **w sobie** — czasy **mnożymy**: O(n) · O(n) = O(n²). Z notatek: O(n²) = O(n · n) = O(n) · O(n).

07 Wykres O(n²)



n	n²	werdykt
10	100	✓ zmieści się
1 000	10^6	✓ zmieści się
10 000	10^8	✓ zmieści się (na styk)
10^5	10^{10}	× za wolno

Wykres to **parabola** — wygina się w górę. 10 razy większe n → **100 razy** więcej operacji.

07 Pętla od i — wciąż O(n²)

```
for(int i = 0; i < n; i++){
    for(int j = i; j < n; j++){
        // O(1)
    }
}
```

i	obrotów j
0	n
1	n - 1
…	…
n - 1	1

Teraz j zaczyna od i, więc pętla wewnętrzna robi się coraz krótsza. Razem:

$$n + (n-1) + \ldots + 1 = n\cdot(n+1)/2$$

> To ten sam wzór co liczba podtablic z lekcji o vectorze! $n\cdot(n+1)/2 = n^2/2 + n/2$. Stałą 1/2 i mniejszy składnik pomijamy → **O(n²)**.

07 O(n³) — trzy pętle w sobie

```
// wypisz wszystkie podtablice (lekcja o vectorze)
for(int i = 0; i < n; i++){           // początek
    for(int j = i; j < n; j++){       // koniec
        for(int k = i; k <= j; k++){  // elementy
            cout << lista[k] << ' ';
        }
        cout << endl;
    }
}
```

Trzy pętle w sobie, każda do (około) n: **O(n) · O(n) · O(n) = O(n³)**.

n	n^3	werdykt
10	1 000	✓ **zmieści się**
100	10^6	✓ **zmieści się**
500	$1{,}25 \cdot 10^8$	✓ **zmieści się** (na styk)
1 000	10^9	× **za wolno**

07 Dwie różne zmienne: n i m

Wejście może mieć **dwie** wielkości, np. n wierszy i m kolumn. Wtedy złożoność zapisujemy z obiema (przykłady z notatek, przepisane na C++):

Kod	Złożoność
`for(i < n) cout << i;`	`O(n)`
`for(i < m) cout << i;`	`O(m)`
`for(i < n) for(j < n) cout << i << j;`	`O(n²)`
`for(i < n) for(j < m) cout << i << j;`	`O(n · m)`
`for(i < n) for(j < m) for(k < n) …`	`O(n² · m)`

Skrót `for(i < n)` oznacza `for(int i = 0; i < n; i++)`. Nie wolno zamienić m na n — to mogą być zupełnie różne liczby (np. n = 1000, m = 3).

07 Cały program — liczy się najwolniejsza część

```
cin >> n;                          // O(1)
for(int i = 0; i < n; i++){        // O(n)
    cin >> lista[i];
}
for(int i = 0; i < n; i++){        // O(n²)
    for(int j = 0; j < n; j++){
        cout << lista[i] + lista[j] << ' ';
    }
}
cout << endl;                      // O(1)
```

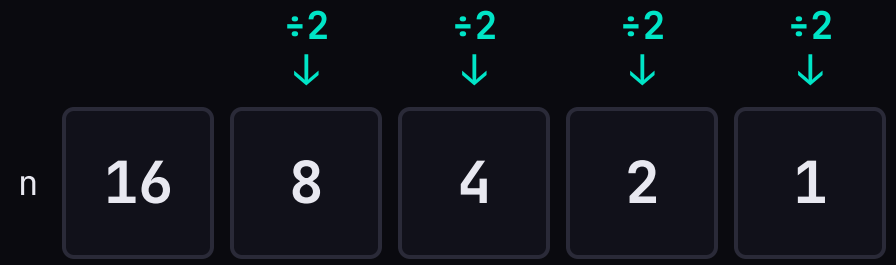
Części stoją po kolei, więc dodajemy:

O(1) + O(n) + O(n²) + O(1) = O(n²)

> Zostaje najwolniejsza część (zasada 2). Program zwalnia tam, gdzie jest najwięcej pracy.

08 Ile razy można podzielić przez 2?

Weźmy liczbę n i dzielmy ją przez 2, aż dostaniemy liczbę **≤ 1**. Ile podzieleń trzeba?



16 → 8 → 4 → 2 → 1 to 4 podzielenia.

Ta liczba podzieleń ma swoją nazwę: **logarytm o podstawie 2** z n. Piszemy **$\log_2 n$**. Czyli $\log_2 16 = 4$.

08 Logarytm — dwie definicje z notatek

Definicja 1: dzielenie

$\log_2 n \approx$ **ile razy trzeba podzielić n na dwa**, aby wyszła liczba ≤ 1.
$\log_2 16$: $16 \to 8 \to 4 \to 2 \to 1 \to$ **4**

Definicja 2: potęgowanie

$\log_2 n \approx$ **najmniejszy x taki, że $2^x \geq n$**.
$\log_2 16$: $2^4 = 16 \geq 16 \to$ **4**

Obie definicje mówią to samo z dwóch stron: **dzielenie przez 2** cofa **mnożenie przez 2**.

	2^1	2^2	2^3	2^4	2^5	2^6	2^7	2^8	2^9	2^{10}
wartość	2	4	8	16	32	64	128	256	512	1024

08 Tabela logarytmów z notatek

n	$\log_2 n$
2	1
4	2
8	3
16	4
32	5
64	6
128	7
256	8
512	9
1024	10

Gdy n jest potęgą dwójki, logarytm to **wykładnik**: $2^{10} = 1024$, więc $\log_2 1024 = 10$.

A gdy n **nie** jest potęgą dwójki? Wtedy logarytm wychodzi „pomiędzy”: $\log_2 1000 \approx 9{,}97$ — trochę mniej niż 10, bo 1000 jest trochę mniejsze od 1024.

> W złożoności nie potrzebujemy dokładnych wartości — wystarczy przybliżenie. Do sprawdzenia dokładnej wartości możesz użyć kalkulatora logarytmów (link w notatkach: calc.pl).

08 Logarytmy dużych liczb

n	10^3	10^4	10^5	10^6	10^7	10^8	10^9
$\log_2 n \approx$	10	13	17	20	23	27	30

Z notatek: $\log_2 10^6 = 19{,}93\ldots \approx$ **20**. Milion trzeba podzielić przez 2 tylko **20** razy!

Zapamiętaj (z notatek): dla liczb do miliarda ($n \le 10^9$) zawsze $\log_2 n \le 30$.

Trik: $2^{10} = 1024 \approx 1000 = 10^3$. Więc co **10 podwojeń** to **×1000**: $\log_2$ tysiąca ≈ 10, miliona ≈ 20, miliarda ≈ 30.

09 Pętla, która dzieli przez 2

Z NOTATEK

```
int n;
cin >> n;
while(n > 1){
    n /= 2;
}
```

`n /= 2` to skrót od `n = n / 2`. Dzielenie intów obcina część po przecinku: 25 / 2 = 12.

obrót	n przed	n po
1	100	50
2	50	25
3	25	12
4	12	6
5	6	3
6	3	1

Dla n = 100 pętla obróciła się tylko **6 razy** ($\log_2 100 \approx 6{,}6$).

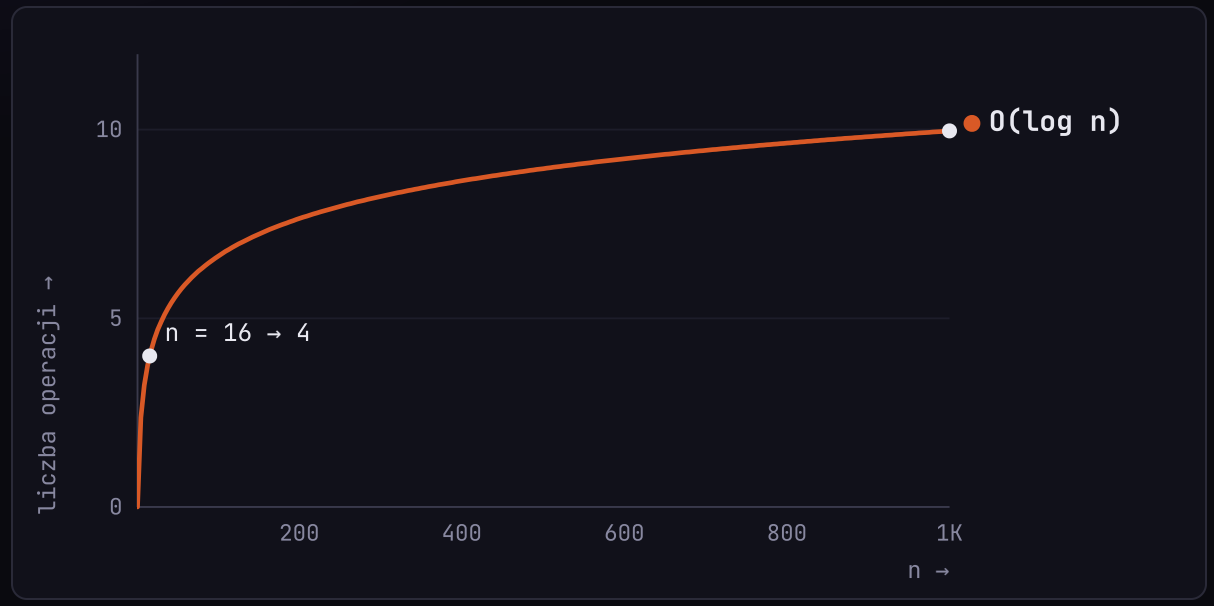
09 Ile obrotów dla dużego n?

```
int n;
cin >> n;
while(n > 1){
    n /= 2;
}
```

n	kolejne wartości n	obrotów
16	16, 8, 4, 2, 1	4
100	100, 50, 25, 12, 6, 3, 1	6
1 000	1000, 500, 250, 125, 62, 31, 15, 7, 3, 1	9
10^6	1000000, 500000, 250000, ..., 3, 1	19
10^9	1000000000, 500000000, ..., 3, 1	29

Wniosek: liczba obrotów ≈ $\log_2 n$. Ta pętla ma złożoność **O(log n)**. Nawet dla miliarda — mniej niż 30 obrotów!

09 Wykres O(log n)



Wykres z notatek: rośnie **bardzo wolno** — prawie płasko.

n	$\log_2 n$
10	3
1 000	10
10^6	20
10^9	30

> Wejście **1000 razy** większe → tylko **+10** operacji. O(log n) to prawie tak dobrze jak O(1).

09 Zabawa: zgadnij liczbę

Myślę o liczbie od **1 do 1000**. Możesz pytać, a ja odpowiadam tylko: **„więcej”** albo **„mniej”**. Ile pytań potrzebujesz?

1. Zawsze pytaj o **środek** zakresu, np. „czy to 500?”.
2. Po każdej odpowiedzi zostaje **połowa** liczb.
3. Zakres dzielisz na pół, aż zostanie jedna liczba.

pytanie	zostaje liczb
start	1000
1	500
2	250
3	125
4	63
…	…
10	1

Wynik: najwyżej **10 pytań** — bo $\log_2 1000 \approx 10$. Dla miliona liczb wystarczy 20 pytań!

09 Uwaga: dzielenie a odejmowanie

O(LOG N)

```
while(n > 1){
    n /= 2;      // dzielimy
}
```

ILE OBROTÓW

dla n = 10^9: 29 obrotów

O(N)

```
while(n > 0){
    n -= 2;      // odejmujemy
}
```

ILE OBROTÓW

dla n = 10^9: 500 000 000 obrotów

Uwaga: obie pętle wyglądają podobnie, ale ta z **odejmowaniem** robi n/2 obrotów — to **O(n)**, a nie O(log n). Logarytm pojawia się tylko, gdy n zmniejsza się **kilka razy** (dzielenie).

09 Dzielenie przez 10 — liczba cyfr

```
int cyfry = 0;
while(n > 0){
    n /= 10;
    cyfry++;
}
cout << cyfry << endl;
```

WEJŚCIE

```
12345
```

NA EKRANIE

```
5
```

n przed	n po
12345	1234
1234	123
123	12
12	1
1	0

Dzielenie przez 10 też jest O(log n) — tylko logarytm ma podstawę 10 (dla miliarda: 10 obrotów). **Podstawa logarytmu to tylko stała**, więc piszemy po prostu O(log n).

10 $O(n \log n) = O(n) \cdot O(\log n)$

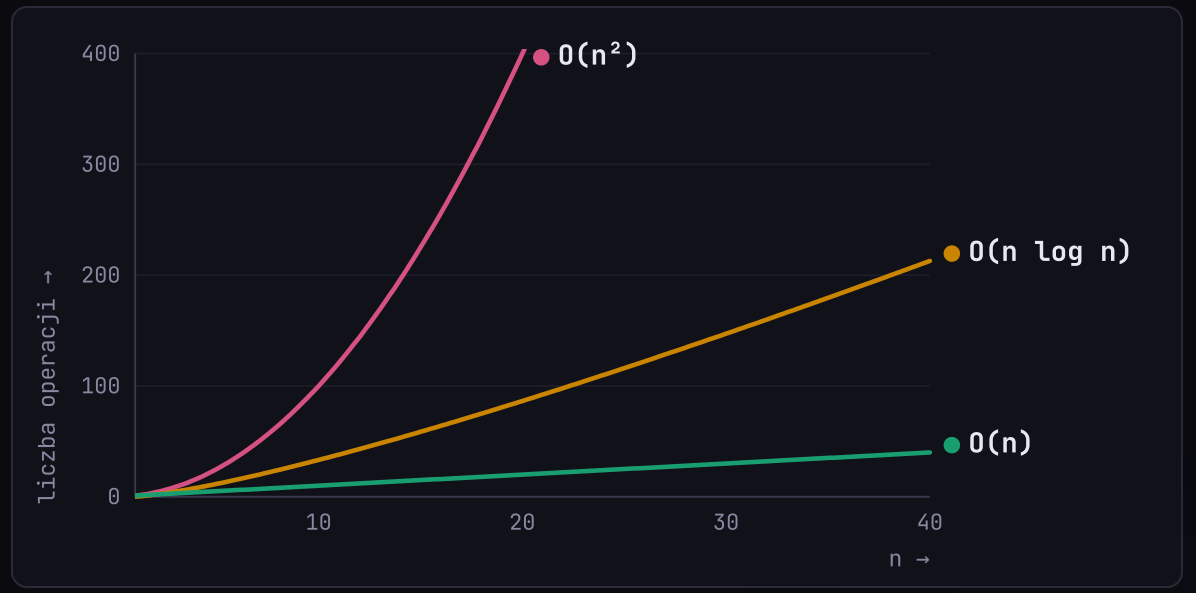
```
for(int i = 0; i < n; i++){       // O(n)
    int x = n;
    while(x > 1){                 // O(log n)
        x /= 2;
    }
}
// razem: O(n) · O(log n) = O(n log n)
```

Z notatek: **$O(n \cdot \log n) = O(n) \cdot O(\log n)$**.

1. Pętla zewnętrzna obraca się **n** razy.
2. W każdym obrocie pętla z dzieleniem obraca się ok. **$\log_2 n$** razy.
3. Pętle są w sobie, więc **mnożymy**: $n \cdot \log n$.

10 Ile to operacji?

n	$n \cdot \log_2 n$	dla porównania n^2
1 000	$1\,000 \cdot 10 = 10^4$	10^6
10^5	$10^5 \cdot 17 \approx 2 \cdot 10^6$	10^{10} **× za wolno**
10^6	$10^6 \cdot 20 = 2 \cdot 10^7$	10^{12} **× za wolno**



O(n log n) jest tylko trochę wolniejsze niż O(n), a dużo szybsze niż O(n²).

10 Gdzie spotkasz O(n log n)?

Sortowanie

`sort(lista.begin(), lista.end());` — funkcja z biblioteki algorithm sortuje vector w czasie **O(n log n)**.

Pętla + dzielenie

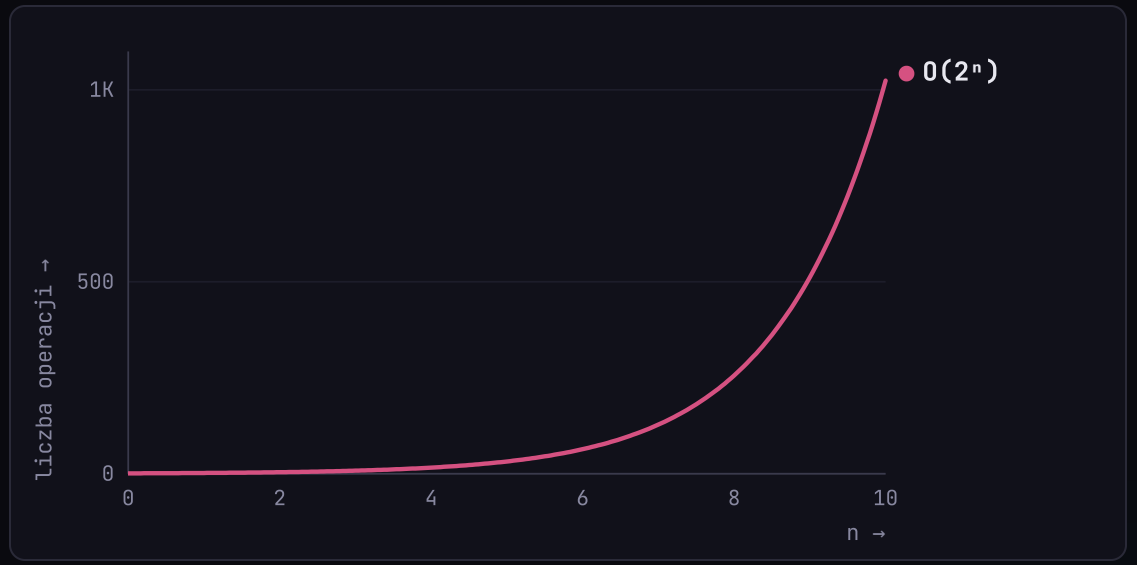
Dla każdego z n elementów liczymy coś pętlą z dzieleniem (np. liczbę cyfr) → n · log n.

Dla $n \le 10^6$ złożoność O(n log n) daje ok. $2 \cdot 10^7$ operacji — mieści się bez problemu. Dlatego to bardzo częsta złożoność rozwiązań na zawodach.

11 O(2ⁿ) — każdy element podwaja pracę

Pamiętasz **podzbiory** z lekcji o vectorze? Przy każdym elemencie decydujemy: bierzemy albo nie. Każdy kolejny element **podwaja** liczbę możliwości.

n	2ⁿ	werdykt
10	1 024	✓ zmieści się
20	ok. 10^6	✓ zmieści się
25	ok. $3 \cdot 10^7$	✓ zmieści się
30	ok. 10^9	× za wolno
60	ok. 10^{18}	× za wolno



Program, który sprawdza **wszystkie podzbiory**, działa w O(2ⁿ). Da się go użyć tylko dla **n ≤ ok. 25**.

11 O(n!) — silnia

Z notatek: **n! = n · (n−1) · (n−2) · … · 3 · 2 · 1**. Czytamy: „en silnia”.

Ile jest sposobów, żeby ustawić **3 osoby** (A, B, C) w kolejce?

ABC	ACB	BAC
BCA	CAB	CBA

3! = 3 · 2 · 1 = **6** — na pierwsze miejsce 3 osoby, na drugie 2, na ostatnie 1.

n	n!	werdykt
3	6	✓ **zmieści się**
5	120	✓ **zmieści się**
10	3 628 800	✓ **zmieści się**
12	ok. $5{\cdot}10^{8}$	**granica**
20	ok. $2{,}4{\cdot}10^{18}$	× **za wolno**

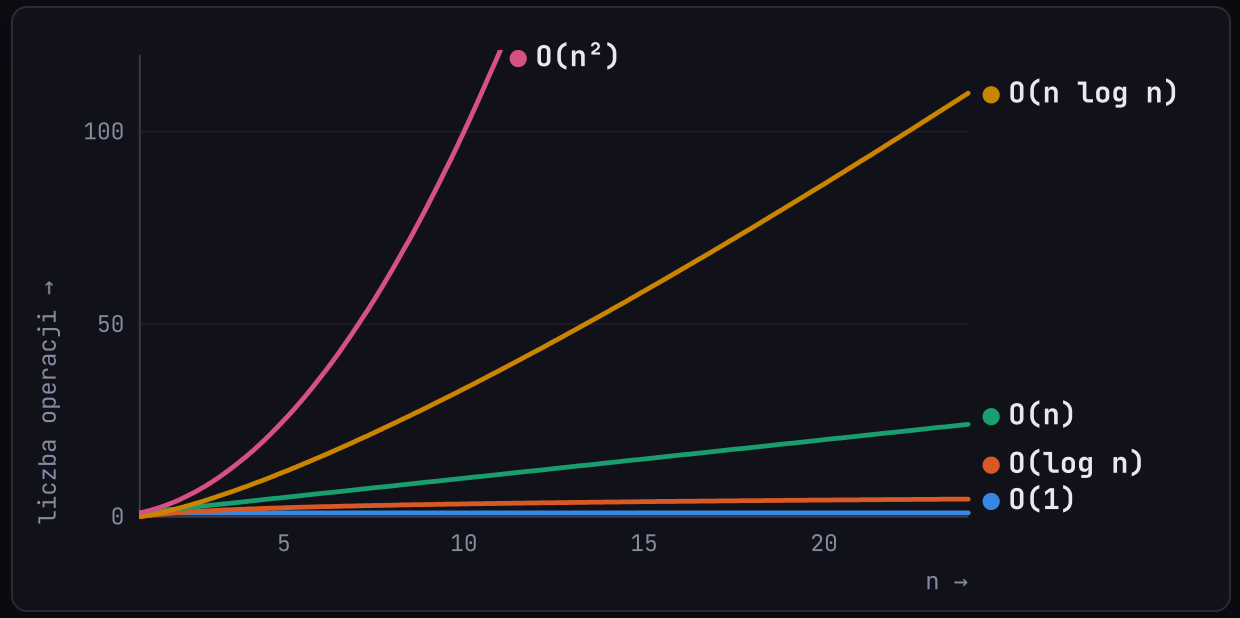
Program, który sprawdza **wszystkie ustawienia** (kolejności), działa w O(n!). Tylko dla **n ≤ ok. 11**.

11 Te złożoności rosną gwałtownie

n	n^2	2^n	n!
5	25	32	120
10	100	1 024	3 628 800
15	225	32 768	ok. $1{,}3 \cdot 10^{12}$
20	400	ok. 10^6	ok. $2{,}4 \cdot 10^{18}$

Uwaga: dla n = 20 złożoność $O(n^2)$ to tylko 400 operacji, a O(n!) — ponad trylion (to **800 lat** liczenia). Złożoności $O(2^n)$ i O(n!) nadają się tylko dla **bardzo małych n**.

12 Wszystkie na jednym wykresie



Każda linia to inna złożoność. Im **wyżej** linia, tym **więcej operacji** i wolniejszy program.

1. O(1) — płasko, nie rośnie wcale.
2. O(log n) — rośnie bardzo powoli.
3. O(n) — prosta linia.
4. O(n log n) — trochę ponad O(n).
5. O(n²) — szybko ucieka w górę.

12 Tabela: ile operacji?

złożoność	n = 10	n = 100	n = 1 000	n = 10^5	n = 10^6
O(1)	1	1	1	1	1
O(log n)	3	7	10	17	20
O(n)	10	100	1 000	10^5	10^6
O(n log n)	33	664	9 966	$1{,}7 \cdot 10^6$	$2{,}0 \cdot 10^7$
O(n^2)	100	10^4	10^6	**10^{10}**	**10^{12}**
O(n^3)	1 000	10^6	**10^9**	**10^{15}**	**10^{18}**
O(2^n)	1 024	**$1{,}3 \cdot 10^{30}$**	**kosmicznie dużo**	**kosmicznie dużo**	**kosmicznie dużo**
O(n!)	$3{,}6 \cdot 10^6$	**kosmicznie dużo**	**kosmicznie dużo**	**kosmicznie dużo**	**kosmicznie dużo**

Pomarańczowe liczby to ponad $5 \cdot 10^8$ operacji — nie zmieszczą się w 1 sekundzie.

12 Od najszybszej do najwolniejszej

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

Znak < czytamy tu „szybsza niż”. Im bardziej na lewo, tym program wytrzyma **większe n**.

Ważne: na zawodach zwykle szukamy rozwiązania, które jest **wystarczająco** szybkie dla największego n z treści. Nie zawsze musi być najszybsze możliwe.

13 Reguły liczenia złożoności

1. Pojedyncze instrukcje (przypisanie, działanie, if, cin, cout jednej liczby) → O(1).
2. Pętla, która obraca się k razy → k · (złożoność wnętrza).
3. Części po kolei (jedna pod drugą) → dodajemy i zostawiamy największą.
4. Pętle w sobie → mnożymy.
5. Pomijamy stałe (3n → n) i mniejsze składniki ($n^2 + n \to n^2$).
6. Pętla, w której n jest dzielone (przez 2, 10, ...) → O(log n).

13 Przykład 1 — kod z notatek

```
int n;                          // O(1)
string napis;                   // O(1)
cin >> n;                       // O(1)
for(int i = 0; i < n; i++){     // n obrotów → O(n)
    cout << i << endl;          // O(1)
}
```

1. Trzy pierwsze linijki: O(1) + O(1) + O(1) = O(1).
2. Pętla: n obrotów · O(1) = O(n).
3. Po kolei: O(1) + O(n) = **O(n)**.

Wynik: O(n)

13 Przykład 2 — dwie pętle po kolei i w sobie

```
for(int i = 0; i < n; i++){          // O(n)
    cout << i << endl;
}
for(int i = 0; i < n; i++){          // O(n)
    for(int j = 0; j < n; j++){      //    · O(n)
        cout << i * j << endl;
    }
}
```

1. Pierwsza pętla: **O(n)**.
2. Druga część to pętla w pętli: O(n) · O(n) = **O(n²)**.
3. Po kolei: O(n) + O(n²) → zostaje **O(n²)**.

Wynik: O(n²)

13 Przykład 3 — n, m i dzielenie

```
for(int i = 0; i < n; i++){          // O(n)
    for(int j = 0; j < m; j++){      //    · O(m)
        cout << i + j << ' ';
    }
}
while(m > 1){                        // O(log m)
    m /= 2;
}
```

1. Pętla w pętli: O(n) · O(m) = **O(n · m)**.
2. Pętla z dzieleniem m: **O(log m)**.
3. Po kolei: O(n · m) + O(log m) → zostaje **O(n · m)**.

Wynik: O(n · m)

14 Czytamy ograniczenia z treści

W treści zadania jest napisane, jak **duże** może być wejście. Przykład z notatek:

Ograniczenia

$1 \le n \le 10^5$

Limit czasu

1 s → ok. 10^8 operacji

1. Bierzemy **największe** możliwe n — tu 10^5 (najgorszy przypadek!).
2. Wstawiamy je do złożoności naszego pomysłu.
3. Porównujemy wynik z 10^8: mniej → zmieści się, dużo więcej → za wolno.

14 Przykład z notatek: $n \le 10^5$

Mamy algorytm	W najgorszym przypadku ($n = 10^5$)	Werdykt
$O(n)$	ok. 10^5 operacji	✓ zmieści się
$O(n^2)$	ok. 10^{10} operacji	× za wolno
$O(1)$	ok. 1 operacja	✓ zmieści się
$O(\log n)$	ok. 17 operacji	✓ zmieści się
$O(n \log n)$	ok. $10^5 \cdot 17 \approx 2 \cdot 10^6$ operacji	✓ zmieści się

W notatkach dla O(log n) jest „ok. 15” — dokładniej $\log_2 10^5 \approx 16{,}6$, czyli ok. 17. Dla werdyktu to bez znaczenia.

14 Jaka złożoność wystarczy?

Ta tabela odpowiada na pytanie: „**jak szybkiego** rozwiązania potrzebuję przy takim n?" (limit 1 s, ok. 10^8 operacji):

Największe n	Wystarczy złożoność	Przykład
$n \le 10$	`O(n!)`	wszystkie ustawienia
$n \le 20$–25	`O(2ⁿ)`	wszystkie podzbiory
$n \le 500$	`O(n³)`	trzy pętle w sobie
$n \le 5\,000$ (do 10 000)	`O(n²)`	pętla w pętli
$n \le 10^6$	`O(n log n)`	sortowanie
$n \le 10^7$–10^8	`O(n)`	jedna pętla
$n \le 10^{18}$	`O(log n)` albo `O(1)`	dzielenie na pół, wzór

14 Ćwiczenie: czy się zmieści?

Limit czasu 1 s (ok. 10^8 operacji). Dla każdego pomysłu powiedz: zmieści się czy nie?

$n \le 1\,000$, pomysł $O(n^2)$

?

$n \le 10^6$, pomysł $O(n^2)$

?

$n \le 20$, pomysł $O(2^n)$

?

$n \le 10^9$, pomysł $O(n)$

?

$n \le 10^9$, pomysł $O(\log n)$

?

$n \le 300$, pomysł $O(n^3)$

?

Policz, **zanim** przejdziesz dalej.

14 Odpowiedzi

$n \le 1\,000$, pomysł $O(n^2)$

ok. 10^6 operacji → ✓ **zmieści się**

$n \le 10^6$, pomysł $O(n^2)$

ok. 10^{12} operacji → × **za wolno**

$n \le 20$, pomysł $O(2^n)$

ok. 10^6 operacji → ✓ **zmieści się**

$n \le 10^9$, pomysł $O(n)$

ok. 10^9 operacji → × **za wolno**

$n \le 10^9$, pomysł $O(\log n)$

ok. 30 operacji → ✓ **zmieści się**

$n \le 300$, pomysł $O(n^3)$

ok. $2{,}7 \cdot 10^7$ operacji → ✓ **zmieści się**

15 Schemat działania na zawodach

Z notatek — jak podchodzić do zadania krok po kroku:

1. **Czytanie. Dokładnie przeczytaj treść, ograniczenia (n!) i limit czasu.**
2. **BRUT — wymyśl coś w miarę prostego, ale wolnego. Ważne, żeby było poprawne.**
3. **WZORCÓWKA — wymyśl coś trudniejszego, ale szybkiego, na 100 punktów.**

Brut (od „brute force” — siłowo) sprawdza wszystko po kolei. **Wzorcówka** (rozwiązanie wzorcowe) ma złożoność dopasowaną do ograniczeń z treści.

15 Po co pisać bruta?

1. Część punktów

Testy często są podzielone na grupy: np. $n \le 1000$ za 30 pkt i $n \le 10^6$ za resztę. Wolny brut zdobywa punkty za małe testy.

2. Sprawdzanie wzorcówki

Na małych testach brut i wzorcówka muszą dać **ten sam** wynik. Jeśli się różnią — szukasz błędu.

3. Zrozumienie zadania

Pisząc bruta lepiej rozumiesz zadanie i łatwiej wpadasz na szybszy pomysł.

4. Spokój

Masz już jakieś punkty, zanim zaczniesz walczyć o 100.

15 Przykład: suma 1 + 2 + … + n, gdy n ≤ 10^9

BRUT – O(N)

```
long long n, suma = 0;
cin >> n;
for(long long i = 1; i <= n; i++){
    suma += i;
}
cout << suma << endl;
```

Dla n = 10^9: miliard obrotów → ok. 10 s → **× za wolno**. Ale dla małych testów (np. n ≤ 10^6) działa i daje punkty.

WZORCÓWKA – O(1)

```
long long n;
cin >> n;
cout << n * (n + 1) / 2 << endl;
```

Dla każdego n: kilka działań → **✓ zmieści się**.

> Na małych n oba programy dają ten sam wynik — tak sprawdzamy, czy wzór jest dobry. Używamy **long long**, bo wynik jest większy niż mieści int.

15 Przykład: ile jest par (i, j), gdzie i < j?

BRUT — O(N²)

```
long long ile = 0;
for(int i = 0; i < n; i++){
    for(int j = i + 1; j < n; j++){
        ile++;
    }
}
cout << ile << endl;
```

WZORCÓWKA — O(1)

```
cout << n * (n - 1) / 2 << endl;
```

n	brut	wzorcówka
4	6	4 · 3 / 2 = 6
1 000	499 500	499 500
10^6	✕ za wolno	✓ zmieści się

Dla i = 0 jest n−1 par, dla i = 1 — n−2, …, razem (n−1) + (n−2) + … + 1 = n·(n−1)/2. Brut pomaga sprawdzić, czy wzór jest dobry.

16 Jaka to złożoność? (1–3)

KOD 1

```
for(int i = 0; i < n; i++){
    for(int k = 0; k < 5; k++){
        cout << i + k;
    }
}
```

KOD 2

```
for(int i = 1; i < n; i *= 2){
    cout << i << ' ';
}
```

KOD 3

```
for(int i = 0; i < n; i++){
    cout << i;
}
for(int i = 0; i < n; i++){
    for(int j = 0; j < n; j++){
        cout << j;
    }
}
```

Zapisz odpowiedzi, **zanim** przejdziesz dalej.

16 Odpowiedzi (1–3)

KOD 1

```
for(int i = 0; i < n; i++){
    for(int k = 0; k < 5; k++){
        cout << i + k;
    }
}
```

O(n) — wewnętrzna pętla zawsze 5 razy → stała

KOD 2

```
for(int i = 1; i < n; i *= 2){
    cout << i << ' ';
}
```

O(log n) — i podwaja się: 1, 2, 4, 8, … — ok. $\log_2 n$ kroków

KOD 3

```
for(int i = 0; i < n; i++){
    cout << i;
}
for(int i = 0; i < n; i++){
    for(int j = 0; j < n; j++){
        cout << j;
    }
}
```

O(n²) — O(n) + O(n²) → zostaje O(n²)

16 Jaka to złożoność? (4–6)

KOD 4

```
while(n > 0){
    n -= 3;
}
```

KOD 5

```
for(int i = 0; i < n; i++){
    for(int j = 0; j < n; j++){
        for(int k = 0; k < m; k++){
            cout << k;
        }
    }
}
```

KOD 6

```
cin >> n;
cout << n * n * n << endl;
```

Zapisz odpowiedzi, **zanim** przejdziesz dalej.

16 Odpowiedzi (4–6)

KOD 4

```
while(n > 0){
    n -= 3;
}
```

O(n) — odejmowanie, nie dzielenie: n / 3 obrotów

KOD 5

```
for(int i = 0; i < n; i++){
    for(int j = 0; j < n; j++){
        for(int k = 0; k < m; k++){
            cout << k;
        }
    }
}
```

O(n² · m) — trzy pętle w sobie: n · n · m

KOD 6

```
cin >> n;
cout << n * n * n << endl;
```

O(1) — n^3 jest w wyniku, ale liczymy go jednym działaniem

16 Najczęstsze błędy

01 **Dodawanie zamiast mnożenia.** Pętle **w sobie** się mnoży (n · n), a dodaje tylko te **po kolei**.

02 **„O(1) to jedna operacja”.** Nie — to **stała** liczba operacji, niezależna od n (może być 100).

03 **Każdy for to O(n).** `for(int i = 0; i < 100; i++)` to O(1), a pętla z `i *= 2` to O(log n).

04 **Odejmowanie jak dzielenie.** `n -= 2` w pętli daje O(n), a `n /= 2` — O(log n).

05 **Zgubiona druga zmienna.** Pętla do m to O(m), nie O(n). O(n · m) to nie to samo co $O(n^2)$.

06 **Liczenie dla małego n.** Złożoność sprawdzamy dla **największego** n z ograniczeń.

16 Ściąga: złożoność czasowa

Złożoność	Typowy kod	Wystarczy dla n ≤	n = 10^5 → operacji
`O(1)`	brak pętli zależnej od n, wzór	dowolnego	1
`O(log n)`	pętla z n /= 2	10^{18}	17
`O(n)`	jedna pętla	10^8	10^5
`O(n log n)`	pętla + dzielenie, sort	10^6	$2 \cdot 10^6$
`O(n²)`	pętla w pętli	5 000	**10^{10}**
`O(n³)`	trzy pętle w sobie	500	**10^{15}**
`O(2ⁿ)`	wszystkie podzbiory	20–25	**kosmicznie dużo**
`O(n!)`	wszystkie ustawienia	10	**kosmicznie dużo**

1 sekunda ≈ 10^8 operacji. Po kolei → dodajemy. W sobie → mnożymy. Stałe pomijamy.

// KONIEC LEKCJI 17

Umiesz już liczyć złożoność czasową!

Wiesz, ile operacji mieści się w sekundzie, umiesz zapisać złożoność w notacji dużego O, policzyć ją z pętli i sprawdzić, czy pomysł zmieści się w limicie dla największego n z treści. Od teraz przy każdym zadaniu: najpierw czytanie, potem brut, a na końcu wzorcówka.